

Software Re-Engineering

A thick, hand-drawn style orange line that spans the width of the title text below it.

Week 3

Program Understanding

- `#include <iostream>`
- `using namespace std;`
- `int x(int A, int B) {`
- `int C = 0, D = 1;`
- `for (int E = 0; E < A; ++E) {`
- `int F = B;`
- `while (F > 1) {`
- `D = D * F;`
- `F -= 2;`
- `}`
- `C += D;`
- `D = 1;`
- `}`
- `return C;`
- `}`
- `void y(int Z[], int n, int W[]) {`
- `for (int H = 0; H < n; ++H) {`
- `int G = Z[H], M = 1;`

- `for (int K = G; K > 0; --K) {`
- `M *= K;`
- `}`
- `W[H] = M;`
- `}`
- `}`
- `int main() {`
- `int P = 3, Q = 5;`
- `int R = x(P, Q);`
- `int S[3] = {R, P, Q};`
- `int T[3];`
- `y(S, 3, T);`
- `for (int N = 0; N < 3; ++N) {`
- `cout << T[N] << endl;`
- `}`
- `return 0;`
- `}`

Program Understanding

- It refers to the process of gaining insight into the **structure**, **functionality**, and **behavior** of a software system, often with the aim of **maintaining**, **improving**, or **extending** it.
- This understanding is crucial for tasks such as **debugging**, **modifying**, **refactoring**, or **adding new features** to existing code.
- It is particularly important in large or complex systems where developers may not have been involved in the original development, or where detailed documentation is lacking.

Key Aspects

- **Code Structure:**
 - Recognizing the high-level design, modules, and organization of the program.
- **Code Behavior:**
 - Understanding how the code functions in different scenarios, including edge cases and real-world inputs.
- **Interdependencies:**
 - Identifying how different parts of the program interact with each other, such as function calls, class relationships, and module dependencies.
- **Documentation and Comments:**
 - Utilizing available documentation or comments to understand the code's intended purpose and logic.
- **Execution Flow:**
 - Analyzing the sequence of operations performed during execution, either statically or dynamically.

Program Understanding Techniques

Top-down
Approach

Bottom-up
Approach

Hypothesis-
Driven
Understanding

Model
Building

Top-down Approach

Top-down Approach

- This approach focuses on understanding a software system from the **highest-level abstractions**
 - Such as the architecture or main modules
- and progressively working down to the **lower-level components**
 - such as functions or individual lines of code.
- This method is useful for understanding the overall design and structure of a program before delving into specific code details.

CUOnline

**COMSATS UNIVERSITY
ISLAMABAD**

 **CUOnline Student Console**

Dear Student,
Display/Wearing of University Student ID Card is mandatory on the campus.

NOTICEBOARD

For expected pass-out Students:
Dear Student, please check carefully your Name, Father's Name, Date of Birth (as per Matric/O-level Certificate), Gender and Photograph appearing on CU-Online are correct.
In case of any discrepancy report at registration@cuiuhore.edu.pk before 25-06-2024, after the deadline changings/corrections will only be made on payment of prescribed fee as per CUI policy.

Information required for "Times Higher Education Impact Ranking 2024"

3 Weeks workshop on Amzon FBA

KAMYAB JAWAN PEPSICO ROSHAN KAL INTERNSHIP PROGRAMME

By Roll NoBy List

CIIT/AA00-BBB-000/LHR

Password

☐ I'm not a robot


reCAPTCHA
Privacy - Terms

Forgot Password ? [Click Here!](#)

[For Parent Console](#)

Login

Top-down Approach - Example

Course
Management

Student
Management

Result
Submission

Database
Layer

Top-down Approach - Example

- Result Submission interacts with the Course Management and Student Management to allow faculty members to submit grades for their respective courses.

```
1  class CourseManagement {
2      public List<Course> getCoursesByInstructor(String instructorId) {
3          // Program Logic
4      }
5  }
6
7  class StudentManagement {
8      public List<Student> getStudentsEnrolledInCourse(String courseId) {
9          // Program Logic
10     }
11 }
12
13 class ResultSubmission {
14     public boolean submitGrades(String courseId, List<Grade> grades) {
15         // Program Logic
16     }
17 }
18
```

Top-down Approach - Example

- You want to modify the submitGrades method to add a feature for “grade moderation,” where faculty can adjust final grades before submission.

```
1  public boolean submitGrades(String courseId, List<Grade> grades) {  
2      for (Grade grade : grades) {  
3            
4              Database.executeUpdate("UPDATE results SET grade = ? WHERE studentId = ? AND courseId = ?",  
5                  grade.getValue(), grade.getStudentId(), courseId);  
6          }  
7      return true;  
8  }  
9
```

Top-down Approach - Example

```
1  public boolean submitGradesWithModeration(String courseId, List<Grade> grades) {
2      for (Grade grade : grades) {
3          Grade moderatedGrade = applyModeration(grade);
4
5
6          Database.executeUpdate("UPDATE results SET grade = ? WHERE studentId = ? AND courseId = ?",
7                                  moderatedGrade.getValue(), moderatedGrade.getStudentId(), courseId);
8      }
9      return true;
10 }
11 private Grade applyModeration(Grade grade) {
12     if (grade.getValue() < 50) {
13         grade.setValue(grade.getValue() * 1.05);
14     }
15     return grade;
16 }
```

Top-down Approach

- That's how a developer using the Top-down Approach would begin with a high-level understanding of the system (e.g., modules, classes) and work down to specific code changes.

Bottom-up Approach

Bottom-up Approach

- The bottom-up approach works in reverse, starting with the analysis of small, well-understood components (e.g., loops, functions or classes) and gradually building an understanding of how these components integrate into the larger system.
- This method is often used when the lower-level code is relatively clear, but the high-level architecture is not well-documented.

Bottom-up Approach

- If one grade fails, the remaining grades still get processed, leading to inconsistent data. This happens because there is no **transaction handling** in place, meaning partial updates can occur.

```
1  public boolean submitGrades(String courseId, List<Grade> grades) {
2      for (Grade grade : grades) {
3
4          Database.executeUpdate("UPDATE results SET grade = ? WHERE studentId = ? AND courseId = ?",
5              grade.getValue(), grade.getStudentId(), courseId);
6      }
7      return true;
8  }
```

Transaction Handling – Solution

- To ensure that all grades are either successfully submitted or none (**atomic transaction**), you update the function to use database transactions.
- This ensures that if one update fails, the system rolls back all changes.

ATOMIC Transaction

- Atomic transactions follow the ACID properties
 - **Atomicity:**
 - The entire transaction is treated as a single unit. If one part of the transaction fails, the whole transaction fails, and the database is rolled back to its previous state.
 - **Consistency:**
 - The transaction ensures that the database moves from one valid state to another, preserving all defined rules and constraints.
 - **Isolation:**
 - Transactions are executed in isolation from one another, preventing interference between transactions.
 - **Durability:**
 - Once a transaction is successfully completed, the changes made are permanent, even in the case of a system failure.

```
1  try {
2      conn.setAutoCommit(false);
3
4      for (Grade grade : grades) {
5
6          stmt.executeUpdate();
7      }
8
9      conn.commit();
10 } catch (SQLException e) {
11     conn.rollback();
12 }
```

Hypothesis-Driven Understanding

Hypothesis-Driven Understanding

- Hypothesis-driven understanding involves forming a hypothesis about how the system works or why a particular issue occurs and testing this hypothesis through experimentation.
 - We are working on a performance issue in the COMSATS Admission Portal hypothesizes that the slow performance is due to inefficient database queries in the “Student Search” feature.
 - We run database profiling to test this hypothesis and confirm that some queries are fetching too much unnecessary data, causing delays.

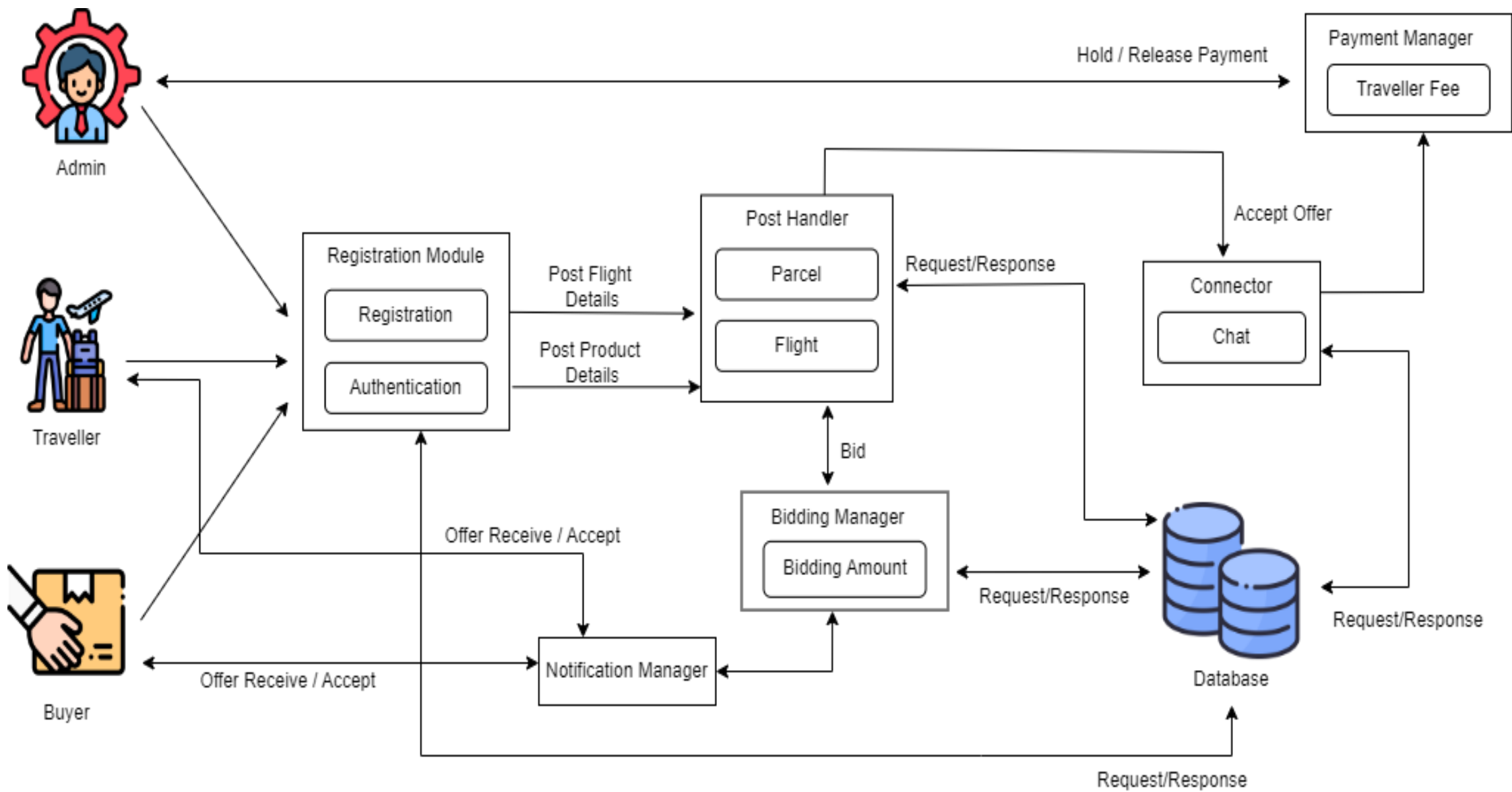
```
1  SELECT *
2  FROM students
3  JOIN courses ON students.course_id = courses.id
4  JOIN grades ON students.id = grades.student_id
5  WHERE courses.name = 'CSE327';
6
```

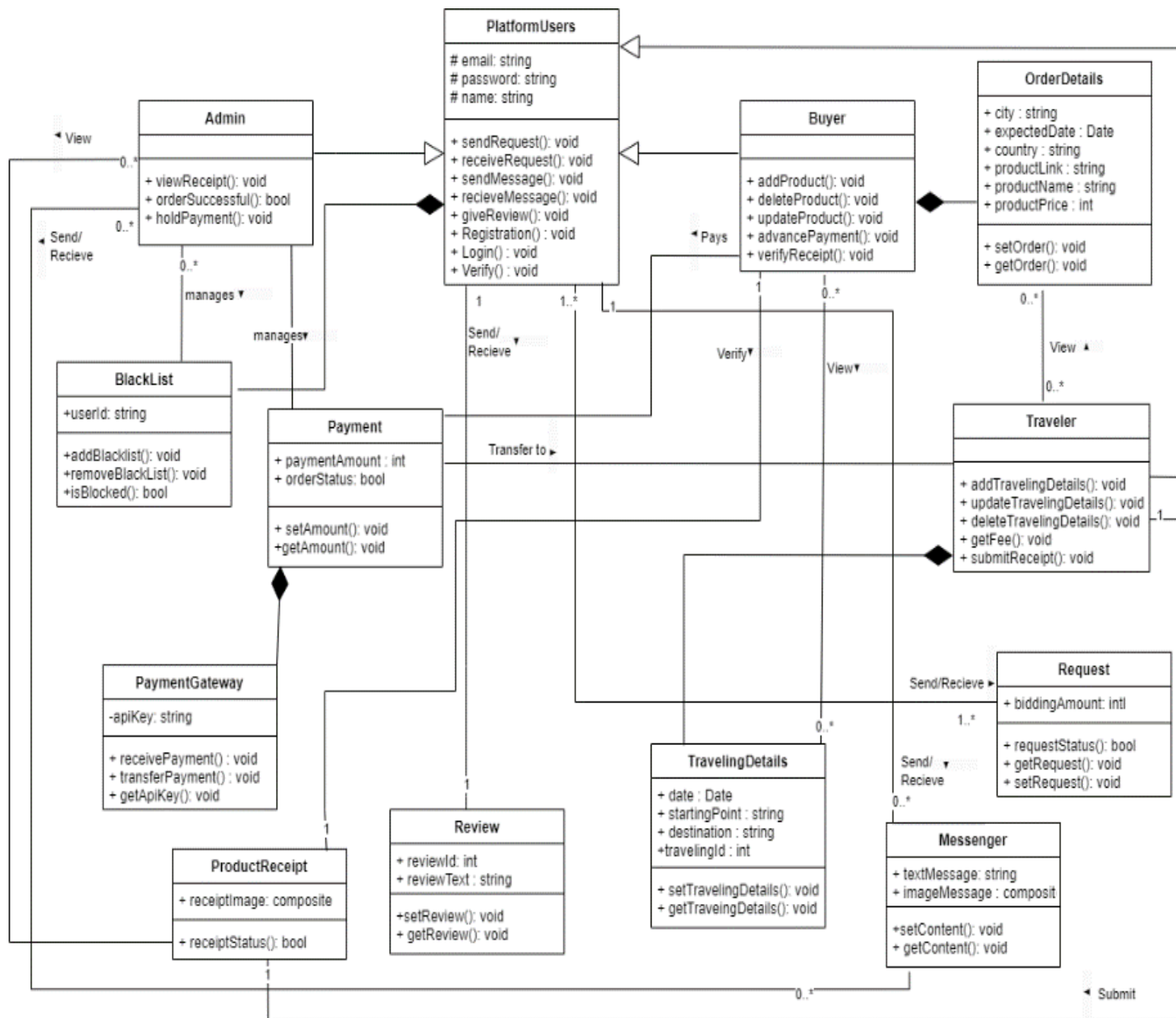
```
1  SELECT students.name, grades.grade
2  FROM students
3  JOIN grades ON students.id = grades.student_id
4  WHERE students.course_id = (SELECT id FROM courses WHERE name = 'CSE327');
5
```


Model Building

Model Building

- Process of creating conceptual or visual representations (models) of a software system.
- These models help developers understand complex systems by breaking them down into manageable parts and visualizing how the components interact.





Analyzing and Understanding Software Documentation

Key Elements of Software Documentation

Code
Comments

API
Documentation

Architectural
Design
Documents

Key Elements of Software Documentation

- Code Comments:
 - Inline comments in code, explaining logic and decision points.
 - Highlight best practices, such as keeping comments up-to-date and concise.
- API Documentation:
 - Explain the importance of well-documented APIs, especially for developers integrating or extending the software.
 - Tools like Swagger or Javadoc helps to generate API documentation.
- Architectural Design Documents:
 - Map out the system's structure, including modules, components, and their interactions.

Challenges in Understanding Software Documentation

Outdated
Documentation

Lack of
Standardization

Complex
Source Code

Challenges in Understanding Software Documentation

- Outdated Documentation:
 - Increased time in troubleshooting and miscommunication among teams.
 - Version control and continuous documentation practices helps us to mitigate this issue.
- Lack of Standardization:
 - Inconsistent documentation styles can confuse developers and reduce productivity.
 - Establishing a standard documentation template or guide within the organization.
- Complex Source Code:
 - Large, complex systems with poor documentation can slow down program understanding and re-engineering efforts.
 - The use of automated tools for documentation generation and keeping documentation closely tied to the source code.

Tools for Software Visualization and Comprehension

- Software visualization refers to the use of visual representations to help developers understand the structure, behavior, and evolution of software systems.
- Visualization aids in identifying patterns, understanding complex architectures, and maintaining code quality.
 - Facilitates communication among developers.
 - Simplifies the understanding of complex systems.
 - Aids in identifying issues early in the development process.

Tools for Software Visualization and Comprehension

Graphviz:

- An open-source tool used for creating graph visualizations from textual descriptions.
- Commonly used for visualizing complex dependency graphs, control flow, and class hierarchies.

Doxygen:

- A documentation generator that produces visual representations of code structures, such as class diagrams and call graphs.
- Widely used for C++, Java, Python, and other languages.

Interactive Debuggers:

- Tools like GDB (GNU Debugger) and Visual Studio Debugger provide real-time visualization of program execution, including memory usage, call stacks, and breakpoints.

Questions?